

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по лабораторной работе №3
“Работа с коллекциями”

A handwritten signature in black ink, likely belonging to the student, is positioned above the student's name.

Выполнил:
Студент группы ИУ5-31Б
Никоноров Д.М.
Преподаватель:
Гапанюк Ю.Е.

Москва 2025

Задание:

Разработать программу, реализующую работу с коллекциями.

1. Программа должна быть разработана в виде консольного приложения на языке C#.
2. Создать объекты классов «Прямоугольник», «Квадрат», «Круг».
3. Для реализации возможности сортировки геометрических фигур для класса «Геометрическая фигура» добавить реализацию интерфейса `Comparable`. Сортировка производится по площади фигуры.
4. Создать коллекцию класса `ArrayList`. Сохранить объекты в коллекцию. Отсортировать коллекцию. Вывести в цикле содержимое коллекции.
5. Создать коллекцию класса `List<Figure>`. Сохранить объекты в коллекцию. Отсортировать коллекцию. Вывести в цикле содержимое коллекции.
6. Модифицировать класс разреженной матрицы (проект `SparseMatrix`) для работы с тремя измерениями – x,y,z. Вывод элементов в методе `ToString()` осуществлять в том виде, который Вы считаете наиболее удобным. Разработать пример использования разреженной матрицы для геометрических фигур.
7. Реализовать класс «`SimpleStack`» на основе односвязного списка. Класс `SimpleStack` наследуется от класса `SimpleList` (проект `SimpleListProject`). Необходимо добавить в класс методы:
 - `public void Push(T element)` – добавление в стек;
 - `public T Pop()` – чтение с удалением из стека.
8. Пример работы класса `SimpleStack` реализовать на основе геометрических фигур.

Листинг кода:

Файлы с реализацией геометрических фигур описаны в предыдущей лабораторной работе не претерпели значительных изменений

Figure.cs

```
using System;
using System.Collections.Generic;
using System.Collections.Specialized;
using System.Runtime.CompilerServices;

namespace Figures
{
    abstract class Figure : Comparable, Comparable<Figure>
    {
        public string Type
        {
            get { return this.type; }
            protected set { this.type = value; }
        }
        string type = "";
    }
}
```

```

        public abstract double Area();

        public int CompareTo(object? obj)
        {
            Figure other = (Figure)obj!;
            if (this.Area() < other.Area()) return -1;
            else if (this.Area() == other.Area()) return 0;
            else return 1;
        }

        public int CompareTo(Figure? obj)
        {
            Figure other = (Figure)obj!;
            if (this.Area() < other.Area()) return -1;
            else if (this.Area() == other.Area()) return 0;
            else return 1;
        }
    }
} // Figures

```

ListNode.cs

```

using System;

namespace Figures
{
    public class ListNode<T> where T : IComparable<T>
    {
        public T data { get; set; }
        public ListNode<T> next { get; set; }

        public ListNode(T param)
        {
            this.data = param;
            this.next = null!;
        }
    }
} // namespace Figures

```

SimpleList.cs

```

using System;
using System.ComponentModel;

namespace Figures
{
    public class SimpleList<T> : IEnumerable<T> where T : IComparable<T>
    {
        private ListNode<T> head = null!;
        private int _count;

        public int Count

```

```

{
    get { return this._count; }
    protected set { this._count = value; }
}

public void Push(T param)
{
    ListNode<T> newElement = new ListNode<T>(param);
    if (head == null)
    {
        this.head = newElement;
    }
    else
    {
        ListNode<T> currentNode = head;
        while (currentNode.next != null)
        {
            currentNode = currentNode.next;
        }
        currentNode.next = newElement;
    }
    _count++;
}

public T Pop()
{
    if (head == null)
    {
        throw new Exception("Cannot pop element from empty list!");
    }

    ListNode<T> currentNode = this.head;
    while (currentNode.next.next != null)
    {
        currentNode.next = currentNode.next.next;
    }
    T lastNodeData = currentNode.next.data;
    currentNode.next.next = null!;
    this.Count--;
    return lastNodeData;
}

public void Sort()
{
    if (head == null || head.next == null)
    {
        return;
    }

    bool swapped;
    do

```

```

    {
        swapped = false;
        ListNode<T> currentNode = head;
        ListNode<T> prevNode = null!;

        while (currentNode != null && currentNode.next != null)
        {
            if (currentNode.data.CompareTo(currentNode.next.data) > 0)
            {
                ListNode<T> sortNext = currentNode.next;
                currentNode.next = sortNext.next;
                sortNext.next = currentNode;
                if (prevNode == null)
                {
                    head = sortNext;
                }
                else
                {
                    prevNode.next = sortNext;
                }
                swapped = true;
            }

            prevNode = currentNode;
            currentNode = currentNode.next;
        }
    } while (swapped);
}

public IEnumerator<T> GetEnumerator()
{
    ListNode<T> current = head;
    while (current != null)
    {
        yield return current.data;
        current = current.next;
    }
}

System.Collections.IEnumerator
System.Collections.IEnumerable.GetEnumerator()
{
    return GetEnumerator();
}
}
}

```

Program.cs

```

using System;
using System.Collections;

```

```

using System.Collections.Generic;
using System.Diagnostics.Contracts;

namespace Figures
{
    class Program
    {
        static void Main(string[] argv)
        {
            //----- Создание объектов геометрических фигур
            Circle circle1 = new Circle(5);
            Circle circle2 = new Circle(155);
            Rectangle rect = new Rectangle(10, 20);
            Square square = new Square(78);

            ArrayList al = new ArrayList();

            al.Add(circle1);
            al.Add(circle2);
            al.Add(square);
            al.Add(rect);

            Console.ForegroundColor = ConsoleColor.Green;
            Console.WriteLine("ArrayList");
            Console.ForegroundColor = ConsoleColor.White;
            foreach (var x in al) Console.WriteLine(x);

            Console.WriteLine("\nОтсортированный список:");
            al.Sort();
            foreach (var x in al) { Console.WriteLine(x); }

            //----- Создание коллекции List из геометрических фигур
            List<Figure> list = new List<Figure>();
            list.Add(circle1);
            list.Add(circle2);
            list.Add(rect);
            list.Add(square);

            //----- Сортировка и вывод на экран
            Console.ForegroundColor = ConsoleColor.Green;
            Console.WriteLine("\nList<Figures>");
            Console.ForegroundColor = ConsoleColor.White;
            Console.WriteLine("Отсортированная коллекция List фигур");
            list.Sort();
            foreach (var x in list) Console.WriteLine(x);

            //----- Создание своей коллекции фигур
            SimpleList<Figure> figures = new SimpleList<Figure>();
            figures.Push(circle1);
            figures.Push(circle2);
            figures.Push(rect);

```

```

        figures.Push(square);

        //----- Сортировка коллекции и вывод на экран
        Console.ForegroundColor = ConsoleColor.Green;
        Console.WriteLine("\nSimpleList<Figure>");
        Console.ForegroundColor = ConsoleColor.White;
        figures.Sort();
        foreach (var x in figures) Console.WriteLine(x);
    }
}
} // Figures

```

Скриншоты работы приложения:

```

● nikod@LAPTOP-752EM865:~/bmstu/Nikonorov_Labs_2025/dotnet$ dotnet run
ArrayList
Circle: r = 5; area: 78.54
Circle: r = 155; area: 75476.76
Square: w = 78; h = 78; area: 6084
Rectangle: w = 10; h = 20; area: 200

Отсортированный список:
Circle: r = 5; area: 78.54
Rectangle: w = 10; h = 20; area: 200
Square: w = 78; h = 78; area: 6084
Circle: r = 155; area: 75476.76

List<Figures>
Отсортированная коллекция List фигур
Circle: r = 5; area: 78.54
Rectangle: w = 10; h = 20; area: 200
Square: w = 78; h = 78; area: 6084
Circle: r = 155; area: 75476.76

SimpleList<Figure>
Circle: r = 5; area: 78.54
Rectangle: w = 10; h = 20; area: 200
Square: w = 78; h = 78; area: 6084
Circle: r = 155; area: 75476.76

```